

Algorithms and Data Structures

02.09.2022

Exercise 1 (1.0 points)

Given the following array of integer values, sort it in ascending order using the merge sort procedure.

13 11 8 6 3 2 9 10 1 15

Display the content of the array just before the last (and conclusive) merge step (the one delivering the final and sorted array). Please, show the entire content of the array as a sequence of integer values separated by a single space. No other symbols must be included in the response.

Exercise 2 (1.0 points)

Consider a binary tree whose visits return the following sequences.

Pre-order:	A	B	G	I	C	E	F	L	D	H
In-order:	I	G	B	A	F	E	L	C	D	H
Post-order:	I	G	B	F	L	E	H	D	C	A

Report the sequence of keys stored on the leaves of the tree, moving on the tree from left to right. Please, report the list of keys on the same line, separated by a single space. No other symbols must be included in the response. This is an example of the response format: A X C Y etc.

Exercise 3 (1.0 points)

A BST contains integer values included in the range 1-1000. Suppose we are looking for the value 0711 in such a BST. Consider the following sequences of values generated during a search.

524 565 626 860 681 785 714 693 699 700 715 713 711
662 938 689 884 691 840 771 717 699 714 707 712 711

Indicate which sequences (numbered as 1 and 2) are correct. Please, report the numbers indicating the correct sequences. These numbers must be separated by a single space and reported in ascending order. No other symbols must be included in the response. This is an example of the response format (when both sequences are correct): 1 2.

Exercise 4 (4.0 points)

First, define the main property which differentiates a binary tree from a BST. Then, suppose a BST storing strings of unknown size is given. Each BST node includes a pointer to the parent and the size of the BST rooted at that node. Write the function to insert a new key into the leaves of the BST. The node type and the function prototypes are the following.

```
typedef struct node_s {
    char *key;
    int size;
    struct node_s *parent;
    struct node_s *left;
    struct node_s *right;
} node_t;

node_t *insert (node_t *root, char *key);
```

Pay particular attention to updating the size of all visited nodes and setting the pointer to the parent of the newly inserted node. If necessary, add parameters to the function insert.

Exercise 5 (1.0 points)

Insert the following sequence of keys into an initially empty hash table. The hash table has a size equal to $M=23$. Insertions occur character by character using open addressing with double hashing. Use functions $h_1(k) = k \% M$ and $h_2(k) = 1 + (k \% 97)$. Each character is identified by its index in the English alphabet (i.e., A=1, ..., Z=26). Equal letters are identified by a different subscript (i.e., A and A become A1 and A2).

A F A F

Indicate in which elements are placed the last two letters of the sequence, i.e., A and F, in this order. Please, report your response as a sequence of integer values separated by one single space. No other symbols must be included in the response. This is an example of the response format: 3 4

Exercise 6 (1.0 points)

Suppose to have an initially empty priority queue implemented with a maximum heap. Consider the following sequence of integers and "*" characters, where each integer corresponds to one insertion into the priority queue and each character "*" corresponds to one extraction.

1 3 8 7 17 13 * *

Report the sequence of values as they are stored in the array representing the priority queue at the end of the entire process. Please, show the entire content of the array as a sequence of integer values separated by a single space. No other symbols must be included in the response. This is an example of the response: 0 3 2 6 8 etc.

Exercise 7 (1.0 points)

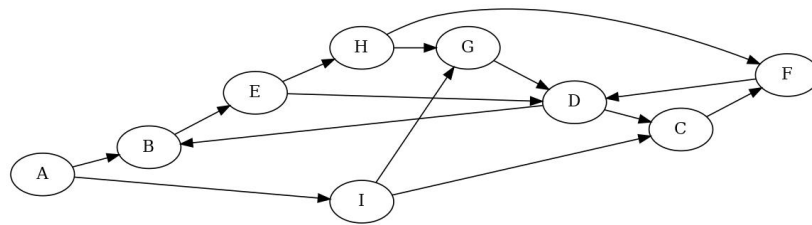
In an activity set, the i -th activity is identified by the pair $[s_i, f_i]$, where s_i is the starting time and f_i is the finishing time. The activities are numbered starting from 1. The following is a correct set of activities.

P1	7	9
P2	17	19
P3	4	6
P4	15	16
P5	6	8
P6	3	7
P7	29	31
P8	28	30
P9	9	12
P10	10	11

Using a greedy algorithm, find the largest subset of mutually compatible activities. Please, report the set of compatible activities in the same order they have been selected separated by a single space. No other symbols must be included in the response. This is an example of the response: 1 3 2 6 8

Exercise 8 (1.0 points)

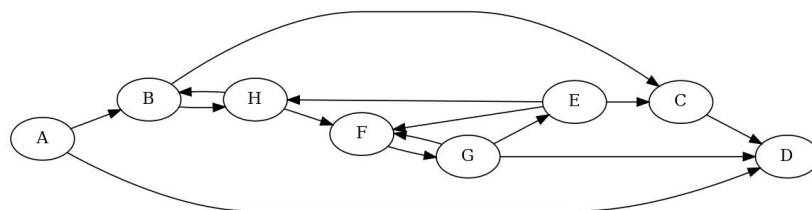
Visit the following graph in depth-first, starting at node A. Label each edge as tree (T), back (B), forward (F), and cross (C). When necessary, consider nodes and edges in alphabetic order.



Report the label of the edges FD, AI, e IC in this order. Please, indicate the edge type of these 3 arcs with a single letter, i.e., T, F, B, C, separated by single spaces. No other symbols must be included in the response. This is an example of the response: B T C

Exercise 9 (1.0 points)

Given the following directed graph, represent the reverse graph and find all strongly connected components.



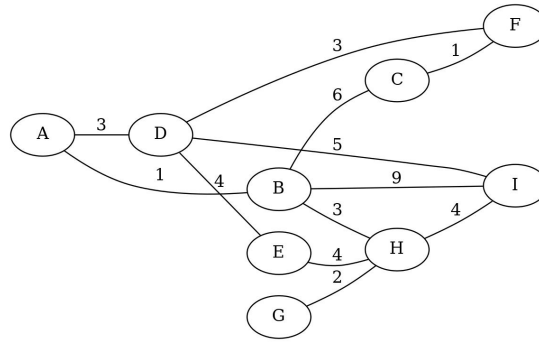
Compute all strongly connected components in the graph. Report them in alphabetic order and within each component indicates vertices in alphabetic order (i.e., ACZ and not CAZ or ZAC). Separate components with a single space. No other symbols must be included in the response. This is an example of the response format: ABC EF XY etc.

Exercise 10 (1.0 points)

Given the following undirected and weighted graph find a minimum spanning tree using Prim algorithm. Start from vertex A.

Indicate the total weight of the final minimum spanning tree. Report one single integer value. No other symbols must be included in the response. This is an example of the response format: 13

Exercise 11 (2.0 points)



Analyze the following program and indicate the exact output generated.

```

int f (int [] [C]);
int main(void) {
    int mat[R][C] = {
        {1,2,3,4,5},
        {2,3,4,5,6},
        {3,4,5,6,7}
    };
    printf ("%d\n", f(mat));
    return (1);
}

int f (int mat[R][C]) {
    int i, j, s, m;
    for (i=0; i<R; i++) {
        s = 0;
        for (j=0; j<C; j++) {
            s = s + mat[i][j];
        }
        if (i==0 || s>m) {
            m = s;
        }
    }
    return (m);
}

```

Report the exact program output with no extra symbols.

Exercise 12 (2.0 points)

Consider the following implementation of heapsort.

```

void heapSort (heap_t *heap) {
    int i, size;
    build_heap (heap);
    size = heap->size;
    for (i=heap->size-1; i>=0; i--) {
        swap (heap, 0, i);
        heap->size--;
        heapify (heap, 0);
    }
    heap->size = size;
    return;
}

static void build_heap (heap_t *heap) {
    int i;
    for (i=(heap->size>>1)-1; i>=0; i--) {
        heapify (heap, i);
    }
    return;
}

static void heapify (heap_t *heap, int i) {
    int l, r, largest;
    l = LEFT(i);
    r = RIGHT(i);
    if ((l<heap->size) && (heap->v[l]>heap->v[i]))
        largest = l;
    else
        largest = i;
    if ((r<heap->size) && (heap->v[r]>heap->v[largest]))
        largest = r;
    if (largest != i) {
        swap (heap, i, largest);
        heapify_max (heap, largest);
    }
    return;
}

```

Indicates which ones of the following statements are correct. Note that incorrect answers imply a penalty in the final score.

1. In function `heapify` the variable `m` indicates the position of the smallest among three elements of the array.
2. The cycle for inside `build_heap` has to go until $i \geq 0$ not until $i > 0$.
3. The variable `size` inside the heap structure must be accessed as `heap->size`, not as `heap->size`, everywhere.
4. The `heapify` procedure has a linear cost.
5. It sorts the array in ascending order.
6. The cycle for inside `heap_sort` must start from `i=heap->size`, not `i=heap->size-1`.

Exercise 13 (2.0 points)

Analyze the following recursive program.

```
void f (int n) {
    int q, r;
    if (n<=0) {
        return;
    }
    q = n/2;
    r = n%2;
    fprintf (stdout, "%d", r);
    f (q);
    return;
}

int main () {
    f(13);
    fprintf (stdout, "\n");
    return 1;
}
```

Report the exact program output with no other symbols.

Exercise 14 (3.0 points)

Write the function

```
void erase_duplicate (char *str);
```

that erase all duplicated characters in the string `str`. More specifically, given the string `str`, the function `erase_duplicate` modifies the same string such that for all characters appearing in the string more than once, only the first occurrence is retained.

For example, if the string `str` initially contains `'aa;;bbbab'` when the function returns, it must contain `'a;b'`. Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Exercise 15 (5.0 points)

Write a C function able to insert some anagraphical data, in the correct position in an ordered list. Data include the surname and name of a person. Both fields are string of characters of maximum length equal to 20.

Use the surname as primary ordering key, and the name as secondary ordering key

The prototype of the function is the following:

```
int order_insert (list_t *list, char *surname, char *name);
```

The function receives the list pointer (of type `list_t`), and the two strings storing the surname and name. The function returns an integer value which is equal to zero when the person is already present in the list, and equal to one if the insertion operation succeeds.

Define the type `list_t` considering that, for each person, surname and name have to be copied in new dynamically allocated and separated fields.

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.

Exercise 16 (9.0 points)

A password is a palindrome string of seven characters, where:

1. The first and seventh characters are the same small letter ($a, \dots, z$).
2. The second and sixth characters are the same capital letter ($A, \dots, Z$).
3. The third and fifth characters are the same digit ($0, \dots, 9$).
4. The fourth character is always a '-' or a '+' character.

For example, passwords `aA1-1Aa`, `cZ9+9Zc`, and `fL0+0Lf` are acceptable, whereas `aA1+1Ab` is not. Write a recursive function to generate all passwords satisfying the previous constraints and write them on standard output.

Write the entire program using standard C libraries but implement all required personal libraries. Modularize the program adequately, and report a brief description of the data structure and the logic adopted in plain English. Unclear or awkward programs, complex or impossible to understand, will be penalized in terms of the final evaluation.